RMFSR training algorithm

Readable mathematical guide / current implementation

This is a typeset version of our implementation notes, checked on 15 October 2026 at update **67,297** of `balanced-v1`. The active data recipe is `figure2-v3-aligned`, with the restored gradual flow schedule and enabled PESQ/SI-SDR waveform losses.

It documents our implementation of the RMFSR method, including our additional losses and reproduction choices. It is not the authors' original training code.

Symbols used throughout

- s: clean target waveform after direct-path mixing and studio processing.
- d: corresponding degraded waveform containing the same underlying speech.
- $c = \mathcal{E}(s)$ and $y = \mathcal{E}(d)$: compressed complex STFT coefficients, stored as real and imaginary channels.
- x_t: intermediate training state; t is current flow time, and $r \le t$ is destination time.
- $F_\theta(x_t, y, t, r)$: CNN prediction in compressed coefficient units.
- $\mathcal{D}$: inverse compression and waveform reconstruction.
- sg: stop-gradient; its argument is constant during parameter backpropagation.

> **Flow time runs from degraded/noisy at $t = 1$ toward clean at $t = 0$.** It is separate from audio time and the optimizer-update counter.

1 Generate a paired training example

Sample one or two EARS speech segments s_j, a shared room and microphone, and separate source positions. Generate full room impulse responses h_j and direct-path counterparts h_j^{dir}. Use the same mixing gains a_j on both branches:

$$s_{\text{dir}} = \sum_{j=1}^{J} a_j (s_j * h_j^{\text{dir}}), \qquad s_{\text{wet}} = \sum_{j=1}^{J} a_j \, \text{EQ}_j(s_j * h_j).$$

$$\begin{aligned} s &= \text{Studio}(s_{\text{dir}}), \\ d &= \text{Corrupt}(\text{Level}(s_{\text{wet}} + \beta \, \text{EQ}_n(n), \ell)) \, . \end{aligned}$$

Here n is DNS background noise, and β sets the sampled SNR. The code draws independent $z_1, z_2 \sim \mathcal{N}(0, 1)$:

$$\text{SNR} = 5 + 10 z_1 \text{ dB}, \qquad \ell = -40 + 10 z_2 \text{ dBFS}.$$

Studio processing: DAPS-reference spectral EQ, mild compression, and a final target level of -25 dBFS. Degradations and reproduction choices continue on the next page.

Data generation: effects and implementation choices

`Corrupt` samples bandwidth limitation, notch filtering, nonlinear distortion, modulation, phase distortion, spectral masking or suppression, MP3/GSM coding, quantization, and 10–80 ms dropouts.

Our choices include a 20% chance of a second source segment, order-2 image-source RIRs, a static soft-knee compressor, and effect probabilities not specified in the paper. Independently sampled segments need not have different speakers. The final damaged level can differ from ℓ because effects follow level normalization.

2 Convert waveforms to coefficients

Use 16 kHz audio, a 320-sample square-root Hann window, and a 160-sample hop: **20 ms frames, 10 ms hop, 161 frequency bins**.

For a frame's Fourier coefficient Z, normalize and compress:

$$\widetilde{Z} = \frac{Z}{\sum_n w[n]}, \qquad \mathcal{E}: \ \widetilde{Z} \mapsto |\widetilde{Z}|^{0.3} e^{i \arg \widetilde{Z}}.$$

Magnitudes are floored near zero for numerical stability. The CNN sees $[B, 2, 161, T]$ tensors; the two channels hold real and imaginary parts. Decoding reverses compression and scale, then uses inverse FFT and weighted overlap-add.

3 Sample flow times and build the state

At zero-based optimizer update k, let $g = k/300000$. Independently for each example, draw

$$t = \max\Big(10^{-4}, \operatorname{sigmoid}(z + 0.4)\Big), \qquad z \sim \mathcal{N}(0, 1),$$

$$r = \begin{cases} t, & \text{with probability } q(g), \\ tU^{\gamma(g)}, & \text{otherwise, with } U \sim \mathcal{U}(0, 1). \end{cases}$$

The current gradual schedules are

$$q(g) = 0.2 + 0.4 \left[1 + \cos\left(\pi \min\left(\frac{g}{0.5}, 1\right)\right)\right],$$
$$\gamma(g) = 0.05 + 0.475 \left[1 - \cos\left(\pi \min\left(\frac{g}{0.8}, 1\right)\right)\right].$$

These approximate the paper's Figure 1 curves. Mapping its epoch axis onto 300,000 updates is our choice; the previously accelerated schedule is no longer active.

Draw pink coefficient noise ϵ: independent Gaussian real/imaginary entries with standard deviation proportional to $1/\sqrt{\max(f, 1)}$, normalized to unit mean variance across frequency bins. Set $\sigma_{\min} = 10^{-8}$ and $\sigma_{\max} = 0.3$:

$$\boxed{x_t = (1 - t)c + ty + \big[(1 - t)\sigma_{\min} + t\sigma_{\max}\big]\epsilon.}$$

The conditional path velocity is

$$v_{\text{cond}} = y - c + (\sigma_{\max} - \sigma_{\min})\epsilon.$$

The clean signal constructs this training state and its supervision. Inference needs only the degraded signal and sampled noise.

4 Compute the MeanFlow target and loss

A. Convert the CNN prediction into a flow

The CNN predicts coefficients. The data-parameterized flow is

$$u_\theta(z, y, t, r) = \frac{z - F_\theta(z, y, t, r)}{t}.$$

B. Obtain the instantaneous tangent

Use a diagonal query, where destination time equals current time, and detach it:

$$v_{\text{diag}} = \text{sg}\left[\frac{x_t - F_\theta(x_t, y, t, t)}{t}\right].$$

C. Compute the directional derivative

Evaluate the Jacobian-vector product at the sampled (x_t, t, r):

$$J_\theta = D_z u_\theta(x_t, y, t, r)[v_{\text{diag}}] + \partial_t u_\theta(x_t, y, t, r).$$

Equivalently, code differentiates with respect to (z, r, t) in direction $(v_{\text{diag}}, 0, 1)$. The degraded condition y and destination r are held fixed.

> **The tangent comes from $r = t$. The differentiated flow is evaluated at the sampled r, t.** These are two distinct network conditions.

D. Construct a stopped coefficient target

$$\begin{aligned} c_\star &= \text{sg}[x_t - t v_{\text{cond}} + t(t - r) J_\theta] \\ &= \text{sg}[c + \sigma_{\min}\epsilon + t(t - r) J_\theta]\,. \end{aligned}$$

E. Minimize the coefficient error

$$\boxed{\mathcal{L}_{\text{MF}} = \text{mean}\left[\left(F_\theta(x_t, y, t, r) - c_\star\right)^2\right].}$$

The mean includes batch, real/imaginary channels, frequencies, and frames. Algebraically, this coefficient MSE equals the mean squared velocity residual weighted by t^2. Replacing it with unweighted velocity MSE changes the objective.

What a small loss means

- **When $r = t$:** the correction vanishes, and $c_\star = c + 10^{-8}\epsilon \approx c$. If the entire microbatch is diagonal, the code skips the unnecessary JVP.
- **When $r < t$:** the target includes the stopped JVP correction. A small main loss is not directly a measurement of final restored-audio error.

5 Add the enabled waveform losses

The additional supervision uses the instantaneous clean estimate:

$$\widehat{c}_{\text{diag}} = F_\theta(x_t, y, t, t), \qquad \widehat{s}_{\text{diag}} = \mathcal{D}(\widehat{c}_{\text{diag}}).$$

This forward pass and decoder retain gradients. It is distinct from the detached diagonal query used for the JVP tangent.

Current total objective

$$\boxed{\begin{aligned}\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{MF}} \\ + 10^{-3}\mathcal{L}_{\text{PESQ}}(s, \widehat{s}_{\text{diag}}) \\ + 10^{-4}\mathcal{L}_{\text{SI-SDR}}(\widehat{s}_{\text{diag}}, s).\end{aligned}}$$

PESQ term

$\mathcal{L}_{\text{PESQ}}$ is the differentiable `torch_pesq.PesqLoss` implementation with internal factor 1.0. It is a training loss, not a reported PESQ score.

SI-SDR term

The term is **negative** SI-SDR, averaged over eligible utterances. Writing $\widehat{s} = \widehat{s}_{\text{diag}}$:

$$\alpha = \frac{\langle \widehat{s}, s \rangle}{\max(\|s\|^2, 10^{-12})},$$
$$\delta = 10^{-8}\max(\|s\|^2, 10^{-12}),$$
$$\mathcal{L}_{\text{SI-SDR}} = -10\log_{10}\frac{\|\alpha s\|^2 + \delta}{\|\widehat{s} - \alpha s\|^2 + \delta}.$$

The code does not subtract waveform means. References with mean-square energy at or below 10^{-12} are excluded from both auxiliary terms. PESQ also excludes predictions below that threshold. Exclusions are logged; an empty eligible set contributes zero for that term.

> **We do not run the multi-step inference solver and backpropagate its final output during training.** These waveform terms supervise a diagonal prediction from the sampled training state.

The waveform losses were added to our run as an adaptation. They are not a claim about the original RMFSR objective. The source paper for this adaptation is linked on page 6.

6 One complete optimizer update

Inputs: CNN parameters θ, EMA parameters $\bar{\theta}$, AdamW state, and update index k. Effective batch size is 16: four microbatches of four examples.

1. Set learning rate $\eta(k)$, diagonal probability $q(k/K)$, and span $\gamma(k/K)$. Clear accumulated parameter gradients.
2. **For each of the four microbatches:**
   a. Generate four target/degraded pairs (s, d) and encode $c = \mathcal{E}(s)$, $y = \mathcal{E}(d)$.
   b. Sample t, r, ϵ independently per example. Construct x_t and v_{cond}.
   c. Construct the stopped MeanFlow target. If all $r = t$, use $c_\star = c + \sigma_{\min}\epsilon$. Otherwise obtain the detached diagonal tangent, compute the JVP, and use $c_\star = \text{sg}[x_t - tv_{\text{cond}} + t(t-r)J_\theta]$.
   d. Predict $p = F_\theta(x_t, y, t, r)$ with parameter gradients. Compute $\mathcal{L}_{\text{MF}} = \text{mean}[(p - c_\star)^2]$.
   e. Obtain a differentiable diagonal estimate $F_\theta(x_t, y, t, t)$, reusing p when the entire microbatch is diagonal. Decode it and calculate PESQ loss and negative SI-SDR.
   f. Form $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{MF}} + 0.001\mathcal{L}_{\text{PESQ}} + 0.0001\mathcal{L}_{\text{SI-SDR}}$. Backpropagate $\mathcal{L}_{\text{total}}/4$ and accumulate gradients.
3. Clip the accumulated global gradient norm to 1.0.
4. Take **one AdamW step**, with weight decay 0.01.
5. Update EMA: $\bar{\theta} \leftarrow 0.999\bar{\theta} + 0.001\theta$.
6. Record losses, update duration, and schedule values. Save or validate when due; increment the completed-update count.

Learning rate and optimizer settings

For $K = 300000$ and zero-based k:

$$\eta(k) = 10^{-4} \min\left(1, \frac{k+1}{5000}\right) \left[0.1 + 0.45\left(1 + \cos(\pi \min(k/K, 1))\right)\right].$$

Each example is a 4-second mono crop. AdamW uses moment coefficients $(0.9, 0.999)$ and numerical epsilon 10^{-8}. The update budget and optimizer settings are our experiment choices.

Checkpoints, validation, and session continuation

- Save every **1,000 updates**, plus improvements and review milestones.
- Validate every **5,000 updates** and at review points: EMA weights, fixed 32-example set, NFE 1, 2, and 5, without backpropagation.
- Select the best checkpoint using **NFE-5 waveform MSE**.
- Six-hour sessions resume model, EMA, optimizer, schedule, and update count.

7 Inference: restore a degraded recording

Given only a degraded waveform d, encode $y = \mathcal{E}(d)$ and initialize

$$x^{(0)} = y + 0.3\epsilon.$$

For N network evaluations, use

$$t_i = 1 - \frac{i}{N}, \qquad r_i = 1 - \frac{i+1}{N}, \qquad i = 0, \ldots, N-1.$$

At each step, predict coefficients and apply the flow update:

$$p_i = F_{\bar{\theta}}(x^{(i)}, y, t_i, r_i),$$

$$\boxed{\begin{aligned} x^{(i+1)} &= x^{(i)} - (t_i - r_i)\frac{x^{(i)} - p_i}{t_i} \\ &= \frac{r_i}{t_i} x^{(i)} + \left(1 - \frac{r_i}{t_i}\right) p_i. \end{aligned}}$$

Return $\widehat{s} = \mathcal{D}(x^{(N)})$. No clean target, training loss, or JVP is used during inference.

> **With NFE 1**, the returned coefficients are exactly $F_{\bar{\theta}}(y + 0.3\epsilon, y, 1, 0)$. With more steps, the final update still returns that step's CNN prediction, evaluated at a different intermediate state.

References and implementation map

Papers

1. RMFSR, Sections 2–3. Core method and data-generation design.
2. Investigating training objectives for flow matching-based speech enhancement, Section 3.4. Basis for our auxiliary waveform-loss adaptation.

Source locations in the project

- Pair generation: `rmfsr/data.py:45`; effects: `rmfsr/degradations.py:82`.
- Transform: `rmfsr/spectral.py:8`.
- Time sampling and MeanFlow loss: `rmfsr/flow.py:29`.
- Waveform losses: `rmfsr/speech_fidelity.py:38`.
- Training loop: `rmfsr/train.py:170`.
- Configuration: `configs/balanced-candidate.json`.

Remote and local versions of `flow.py`, `train.py`, `data.py`, `flow_curriculum.py`, and `speech_fidelity.py` had matching SHA-256 hashes when the source notes were prepared. Earlier checkpoints used different recipes. This document describes that verified training phase, not a fresh live-status check.